

Lineare Algebra - Übungen 7

Klaus Rheinberger, FH Vorarlberg

8. April 2025

i Abgabe in ILIAS

- Alle PYTHON-Aufgaben gesamthaft in einem lauffähigen Jupyter Notebook, siehe Vorlage in ILIAS.
- Alle PAPIER-Aufgaben gesamthaft in einer PDF-Datei inkl. Name auf den gescannten Seiten.

1 Diskretes Dynamische System einer (Vor-)Stadtbevölkerung (1 + 2 + 3 = 6 Punkte)

Jedes Jahr ziehen etwa 10 % der Bevölkerung einer Stadt in die umliegenden Vororte, und etwa 20 % der Vorstadtbevölkerung in die Stadt. Im Jahr 2024 gibt es 10 Millionen Menschen in der Stadt und 8 Millionen Menschen in den Vororten, was wir mit dem Zustandsvektor $x_0 = \begin{pmatrix} 10 \\ 8 \end{pmatrix}$ modellieren.

1. PAPIER: Bestimmen Sie die Übergangsmatrix M , so dass Mx der Einwohnerverteilung ein Jahr nach x entspricht.
2. PAPIER: Analysieren Sie das Langzeitverhalten des dynamischen Systems mit Hilfe der Eigenwerte und -vektoren von M , d. h. wohin tendiert der Zustandsvektor für große Zeiten?
3. PYTHON: Überprüfen Sie Ihr Ergebnis, indem Sie
 1. EW und EV in Python nachrechnen,
 2. eine Simulation der Dynamik für 50 Jahre durchführen und diese grafisch darstellen,
 3. den Anfangszustand x_0 in der Basis der Eigenvektoren darstellen und so die Langzeitdynamik analysieren.

Lösung: *Quelle:* David Lay, Linear Algebra and Its Applications, 4th edition, p.87, exercise 10

Die Übergangsmatrix ist $M = \begin{pmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{pmatrix}$ und hat die Eigenwerte $\lambda_1 = 1$ und $\lambda_2 = 0.7$ mit Eigenvektoren $v_1 = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}$ und $v_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$. Der Anfangszustand $x_0 = \begin{pmatrix} 10 \\ 8 \end{pmatrix} = 18 \cdot v_1 + 2 \cdot v_2$ tendiert daher für große Zeiten gegen $x = 18 \cdot v_1 = \begin{pmatrix} 12 \\ 6 \end{pmatrix}$.

Genauere Beschreibung der Dynamik der langfristigen Zustandsentwicklung: Sei x_t der Zustand nach t Jahren. Dann gilt

$$\begin{aligned}x_t &= M^t x_0 \\&= M^t (18e_0 + 2e_1) \\&= 18M^t e_0 + 2M^t e_1 \\&= 18 \cdot 1^t \cdot e_0 + 2 \cdot 0.7^t \cdot e_1\end{aligned}$$

Somit gilt $\lim_{t \rightarrow \infty} x_t = \lim_{t \rightarrow \infty} M^t x_0 = 18e_0 = (12, 6)^T$.

```
# initial state:
x0 = np.array([[10],
               [8]])

# transition matrix:
M = np.array([[0.9, 0.2],
              [0.1, 0.8]])

# dynamics:
n = 50 # number of years
times = np.arange(n) # vector of years from now
X = np.zeros((2, n)) # matrix with states as columns

X[:,0] = x0 # put initial state into first column
for k in range(1, n): # iterate over years
    X[:,k] = M@X[:,k-1]

plt.figure(figsize=(5, 3))
plt.plot(times, X[0,:], '.-', label='Stadt')
plt.plot(times, X[1,:], '.-', label='Vororte')
plt.legend(loc='best')
plt.xlabel('Zeit [Jahr]')
plt.ylabel('Anzahl Menschen [Mio.]')
plt.ylim(0,13)
plt.grid(True)

# eigenvalues and eigenvectors:
L, V = np.linalg.eig(M)
print(f"eigenvalues = {L}")
print(f"matrix with eigenvectors in columns = \n{V}")

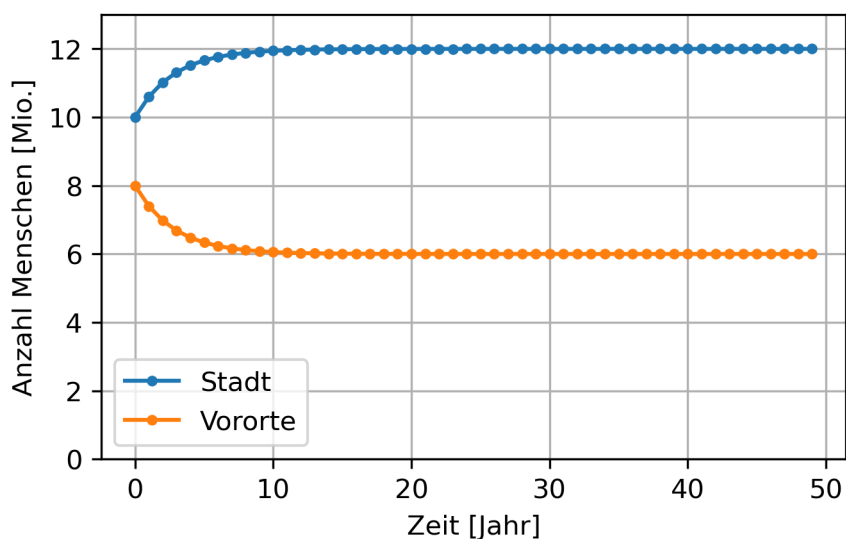
# rescaling of eigenvectors:
e0 = V[:,0] # first column
e0 = e0/sum(e0)
e1 = np.array([[-1],
               [ 1]])
E = np.hstack([e0, e1])
print(f"matrix with rescaled eigenvectors in columns = \n{E}")

# representation of the initial state x0 as a linear combination of the eigenvectors:
# y00*e0 + y10*e1 = E*y0 = x0, with y0 the column vector of the coefficients
y0 = np.linalg.solve(E, x0)
```

```
print(f"y0 = \n{y0}")

# The part of the linear combination belonging to eigenvalue 1 dominates.
# and forms the limit value for large times, see figure above.:
print(f"limit value = \n{y0[0,0]*e0}")
```

```
eigenvalues = [1. 0.7]
matrix with eigenvectors in columns =
[[ 0.89442719 -0.70710678]
 [ 0.4472136  0.70710678]]
matrix with rescaled eigenvectors in columns =
[[ 0.66666667 -1.      ]
 [ 0.33333333  1.      ]]
y0 =
[[18.]
 [ 2.]]
limit value =
[[12.]
 [ 6.]]
```



Endergebnis

Der Zustand tendiert für große Zeiten gegen $x = \begin{pmatrix} 12 \\ 6 \end{pmatrix}$.

2 Ein Räuber-Beute-System (2 + 2 = 4 Punkte)

Tief in den Redwood-Wäldern von Kalifornien liefern die [Dunkelfuß-Buschratten](#) bis zu 80 % der Nahrung für den [Fleckenkauz](#), den Hauptfeind der Buschratten. Wir verwenden ein lineares dynamisches System, um die Dynamik des Eulen- und Rattenbestands zu modellieren. Das Modell ist in

mehrfacher Hinsicht unrealistisch, aber es kann als Ausgangspunkt für die Untersuchung komplizierterer nichtlinearer Modelle verwendet werden. Mit $x_t = \begin{pmatrix} f_t \\ b_t \end{pmatrix}$ bezeichnen wir für den Monat t den Bestand des Fleckenkaux sowie den Bestand an Buschratten, letztere gemessen in Tausenden. Ein Monat später sei der Bestand gegeben durch

$$\begin{aligned} f_{t+1} &= 0.5f_t + 0.4b_t, \\ b_{t+1} &= -0.104f_t + 1.1b_t. \end{aligned}$$

Der Term $0.5f_t$ in der ersten Gleichung besagt, dass ohne Buschratten als Nahrung nur die Hälfte der Fleckenkaux jeden Monat überleben wird, während der Term $1.1b_t$ in der zweiten Gleichung besagt, dass ohne Fleckenkaux als Räuber die Buschrattenpopulation um 10 % pro Monat zunehmen wird. Wenn es viele Buschratten gibt, führt der Term $0.4b_t$ zu einem Anstieg der Fleckenkauxpopulation, während der negative Term $-0.104f_t$ den Tod von Buschratten durch den Fleckenkaux beschreibt. Ein Fleckenkaux frisst also pro Monat im Schnitt 104 Buschratten.

Wir beschreiben die Bestandsdynamik kann durch die Vektorgleichung $x_{t+1} = Mx_t$. Die Übergangsmatrix M lautet

$$M = \begin{pmatrix} 0.5 & 0.4 \\ -0.104 & 1.1 \end{pmatrix}.$$

1. PYTHON: Bestimmen Sie die Eigenwerte und Eigenvektoren von M . Skalieren Sie die Eigenvektoren sinnvoll.
2. PAPIER: Bestimmen Sie daraus den langfristigen Bestand der Eulen und Buschratten.
3. PYTHON: Plotten Sie die Bestandsentwicklung der Eulen und Buschratten für die nächsten Monate für unterschiedliche Anfangsbedingungen in der f - b -Ebene.

Lösung: *Quelle:* David Lay, Linear Algebra and Its Applications, 6th edition, p. 336f

```
M = np.array([[ 0.5,  0.4],
               [-0.104, 1.1]])
[L, V] = np.linalg.eig(M)
print(f"eigenvalues = {L}")
print(f"matrix with eigenvectors in columns = \n{V}")

# rescaling of eigenvectors:
v1 = V[:, [0]]/V[0, 0]
v2 = V[:, [1]]/V[0, 1]
print(f"rescaled 1st eigenvector: \n{v1}")
print(f"rescaled 2nd eigenvector: \n{v2}")
```

```
eigenvalues = [0.58 1.02]
matrix with eigenvectors in columns =
[[-0.98058068 -0.60971076]
 [-0.19611614 -0.79262399]]
rescaled 1st eigenvector:
[[1. ]
 [0.2]]
rescaled 2nd eigenvector:
[[1. ]
 [1.3]]
```

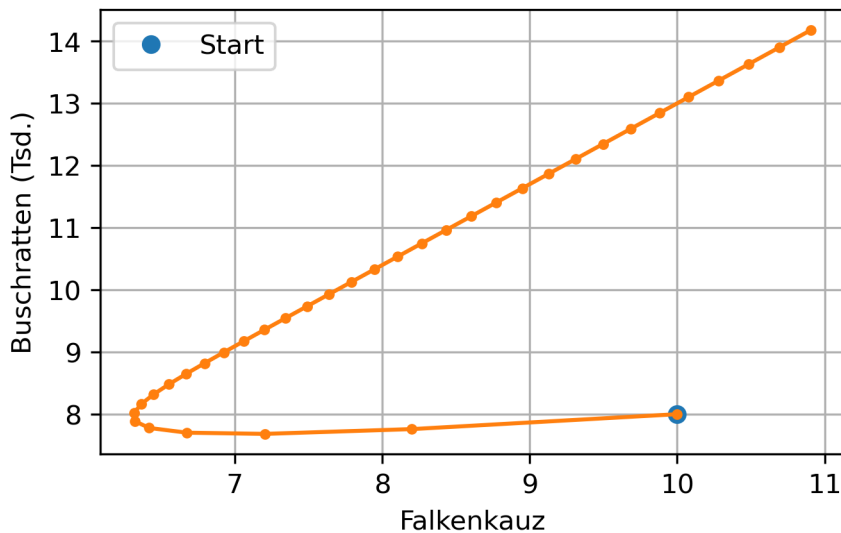
Jeder Anfangszustand x_0 kann in der Eigenbasis dargestellt werden: $x_0 = c_1 v_1 + c_2 v_2$, wobei v_1 und v_2 die Eigenvektoren sind und c_1 und c_2 die Koordinaten von x_0 in der Eigenbasis. Der Bestand n Monate später ist dann

$$\begin{aligned} x_t &= M^t x_0 \\ &= c_1 M^t v_1 + c_2 M^t v_2 \\ &= c_1 \lambda_1^t v_1 + c_2 \lambda_2^t v_2 \\ &= c_1 0.58^t v_1 + c_2 1.02^t v_2. \end{aligned}$$

Der erste Term $c_1 0.58^t v_1$ geht mit der Zeit t rasch gegen Null, während der zweite Term $c_2 1.02^t v_2$ gegen den langfristigen Bestand konvergiert. Der langfristige Bestand ist also proportional zum Eigenvektor $v_2 = \begin{pmatrix} 1 \\ 1.3 \end{pmatrix}$ mit dem größeren Eigenwert $\lambda_2 = 1.02$. Langfristig gibt es pro Fleckenzauz 1300 Buschratten.

```
# initial state:
x0 = np.array([[10],      # Falkenzauz
               [ 8]])     # Buschratten (Tsd.)
T = 36 # Anzahl der Monate
x_ = np.zeros((2, T))
x_[:,0] = x0
for t in range(1, 36):
    x_[:,t] = M @ x_[:,t-1]

plt.figure(figsize=(5, 3))
plt.plot(x0[0], x0[1], 'o', label='Start')
plt.plot(x_[0,:], x_[1,:], '.-')
plt.xlabel('Falkenzauz')
plt.ylabel('Buschratten (Tsd.)')
plt.legend(loc='best')
plt.grid(True)
```



Endergebnis

1. Die Eigenwerte sind $\lambda_1 = 0.58$ und $\lambda_2 = 1.02$. Die Eigenvektoren sind z. B. $v_1 = \begin{pmatrix} 1 \\ 0.2 \end{pmatrix}$ und $v_2 = \begin{pmatrix} 1 \\ 1.3 \end{pmatrix}$.
2. Langfristig gibt es pro Fleckenkauz 1300 Buschratten.

3 Müllmengen (2 + 3 = 5 Punkte)

Die Müllmenge in Millionen Tonnen pro Tag des Landes [Lower Slobbovia](#) von 1960 bis 1995 ist in folgender Tabelle wiedergegeben.

Jahr t	1960	1965	1970	1975	1980	1985	1990	1995
Menge y	86.0	99.8	135.8	155.0	192.6	243.1	316.3	469.5

1. PAPIER: Formulieren Sie das OLS-Problem in Matrixform für den besten Fit einer Gerade durch die Datenpunkte.
2. PYTHON: Plotten Sie die Daten, lösen Sie das OLS-Problem, und stellen Sie Daten und optimalen Fit in einem Plot grafisch dar.

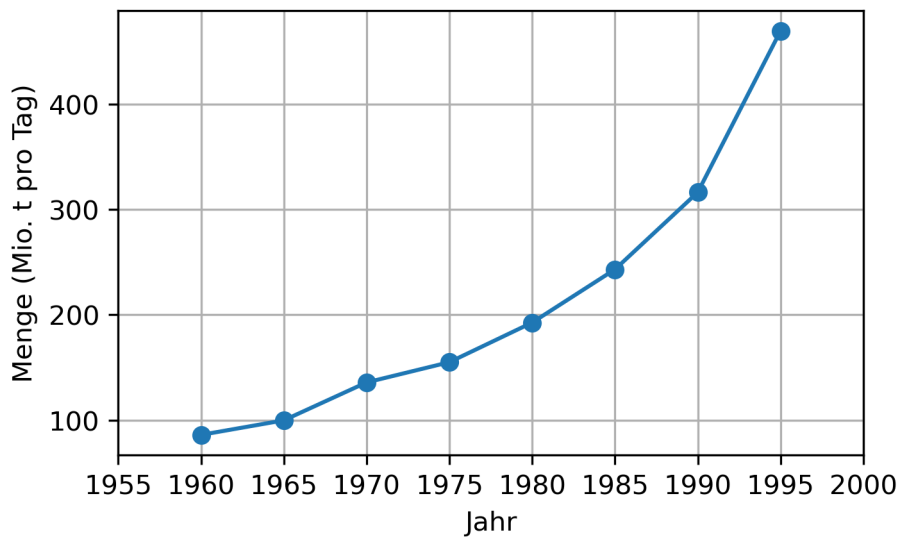
Lösung:

Minimiere $\sum_{t=1960}^{1995} (y_t - (d + kt))^2$ in Matrixform:

$$\min \left\| \begin{pmatrix} 1 & 1960 \\ 1 & 1965 \\ \vdots & \vdots \\ 1 & 1995 \end{pmatrix} \begin{pmatrix} d \\ k \end{pmatrix} - \begin{pmatrix} 86.0 \\ 99.8 \\ \vdots \\ 469.5 \end{pmatrix} \right\|$$

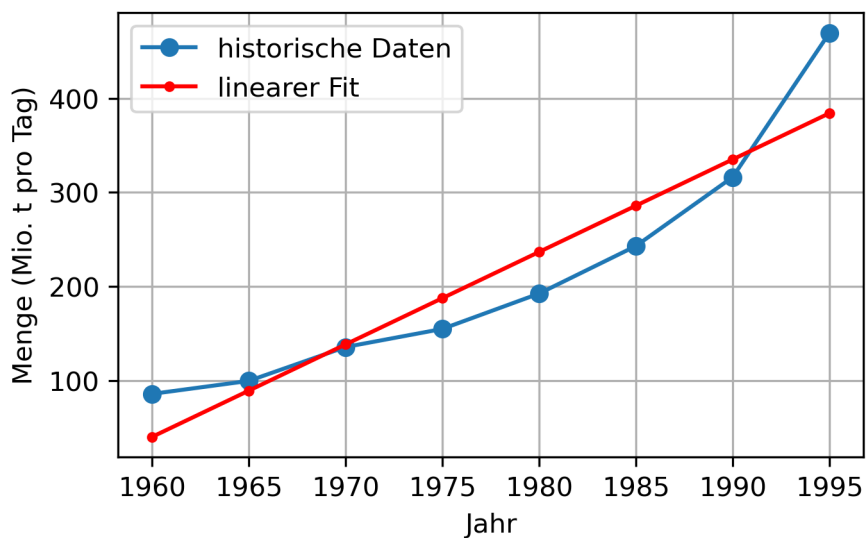
```
t = np.arange(1960, 2000, 5)
y = np.array([86.0, 99.8, 135.8, 155.0, 192.6, 243.1, 316.3, 469.5])

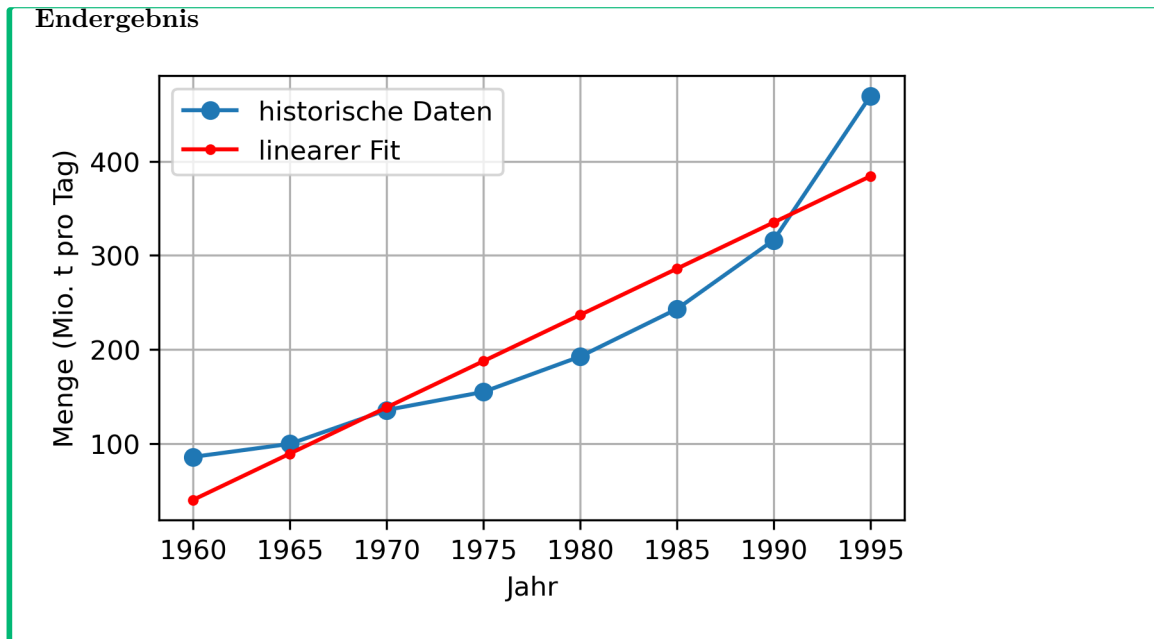
plt.figure(figsize=(5, 3))
plt.plot(t, y, 'o-')
plt.xlim(1955, 2000)
plt.xlabel('Jahr')
plt.ylabel('Menge (Mio. t pro Tag)')
plt.grid(True)
```



```
# linear, d. h. Gerade:
n = len(t)
A = np.column_stack( (np.ones((n, 1)), t) )
x_linear = np.linalg.lstsq(A, y)[0]
y_fit_linear = A @ x_linear

plt.figure(figsize=(5, 3))
plt.plot(t, y, 'o-', label='historische Daten')
plt.plot(t, y_fit_linear, '-r', label='linearer Fit')
plt.xlabel('Jahr')
plt.ylabel('Menge (Mio. t pro Tag)')
plt.legend(loc='best', numpoints=1)
plt.grid(True)
```





4 Ohmsches Gesetz (1 + 2 + 2 + 1 = 6 Punkte)

Sie haben in einer Messreihe die Spannung U und den Strom I an einem Ohmschen Widerstand gemessen und die folgenden Daten erhalten:

I (A)	0.11	0.15	0.27	0.40	0.50	0.54	0.63	0.89	0.99	1.10
U (V)	1.10	2.08	2.94	4.39	4.87	5.99	7.70	8.30	8.50	10.03

1. PYTHON: Stellen Sie die Daten grafisch dar.
2. PYTHON: Schätzen Sie den Ohmschen Widerstand R , indem Sie mit dem Ohmschen Gesetz $U = R \cdot I$ und einer linearen Regression, die den quadratische U -Fehler minimiert, den optimalen Widerstandswert bestimmen.
3. PYTHON: Schätzen Sie nun den Widerstand R mit einer linearen Regression, die den quadratische I -Fehler minimiert.
4. PAPIER: Vergleichen Sie die beiden Schätzungen. Welche Methode ist besser? Unter welchen Bedingung liefern die beiden Methoden den selben Schätzwert für R ?

Lösung: Siehe Code. Da die A -Matrix in beiden Fällen nur aus einer Spalte besteht, ist der optimale Fit bzgl. $Ax = b$ die orthogonale Projektion des b -Vektors auf die Spalte von A . In der zweiten Schätzung sind A und b im Vergleich zur ersten Schätzung vertauscht. In der zweiten Schätzung wird daher gerade die andere orthogonale Projektion verwendet. Die beiden Projektionen sind nur identisch, wenn die Vektoren parallel sind, d. h. die Datenpunkte auf einer Geraden liegen. Die beiden Schätzungen sind also genau dann identisch, wenn die Datenpunkte auf einer Geraden liegen.

```
# data as column vectors:
I = np.array([[0.11, 0.15, 0.27, 0.40, 0.50, 0.54, 0.63, 0.89, 0.99, 1.10]]).T
U = np.array([[1.10, 2.08, 2.94, 4.39, 4.87, 5.99, 7.70, 8.30, 8.50, 10.03]]).T
```



```

plt.figure(figsize=(5, 3))
plt.plot(I, U, 'o')
plt.xlabel('I (A)')
plt.ylabel('U (V)')
plt.xlim(0, 1.2)
plt.ylim(0, 12)
plt.grid(True)

print("IR = U:")
A = I
b = U
# Via Projektion auf die Spalte von A:
x = 1/(A.T@A)*A.T@b
R1 = x[0,0]
print(f"R = {R1:.2f} Ohm")
# Via OLS:
x = np.linalg.lstsq(A, b)[0]
print(f"R = {x[0,0]:.2f} Ohm")

print("U*1/R = I:")
A = U
b = I
# Via Projektion auf die Spalte von A:
x = 1/(A.T@A)*A.T@b
R2 = 1/x[0,0]
print(f"R = {R2:.2f} Ohm")
# Via OLS:
x = np.linalg.lstsq(A, b)[0]
print(f"R = {1/x[0,0]:.2f} Ohm")

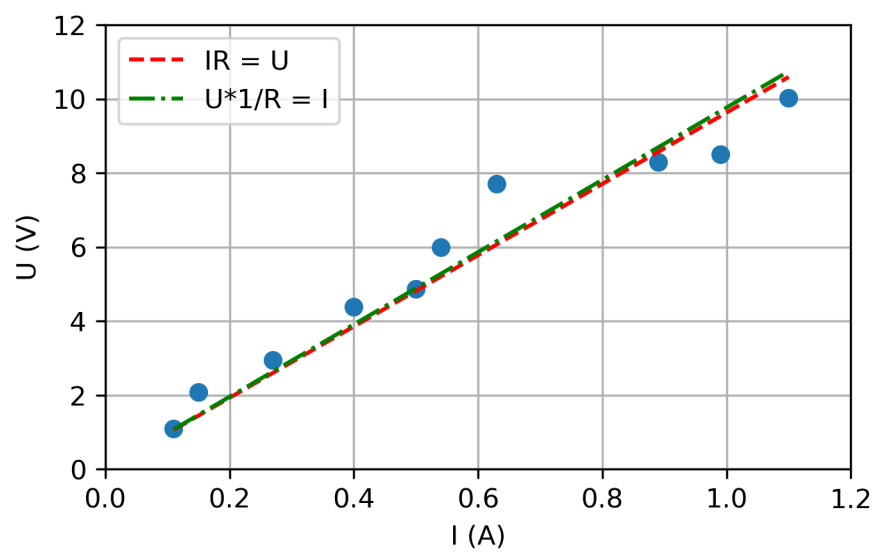
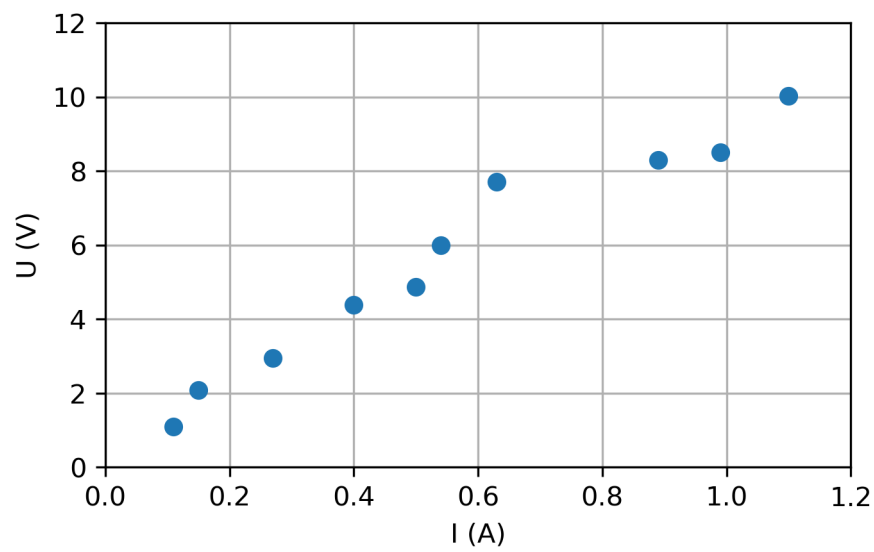
plt.figure(figsize=(5, 3))
plt.plot(I, U, 'o')
plt.plot(I, I*R1, '--r', label='IR = U')
plt.plot(I, I*R2, '-.g', label='U*1/R = I')
plt.xlabel('I (A)')
plt.ylabel('U (V)')
plt.xlim(0, 1.2)
plt.ylim(0, 12)
plt.legend(loc='best')
plt.grid(True)

```

```

IR = U:
R = 9.63 Ohm
R = 9.63 Ohm
U*1/R = I:
R = 9.77 Ohm
R = 9.77 Ohm

```



Endergebnis

$IR = U$:
 $R = 9.63 \text{ Ohm}$
 $U \cdot 1/R = I$:
 $R = 9.77 \text{ Ohm}$

